

Deep Learning

Lecture 9: Recurrent Networks

Room: **BIN-1-B.01**

Instructor: Manuel Günther

Email: guenther@ifi.uzh.ch

Office: AND 2.54

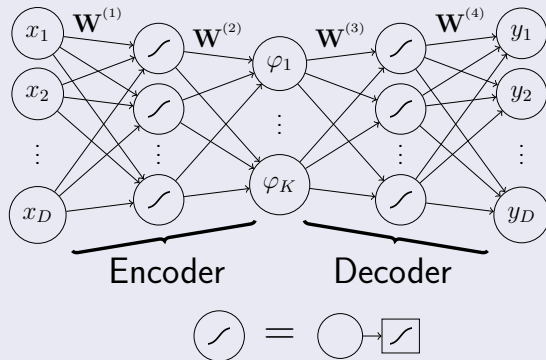
Friday, May 8, 2026, 8:15 – 10:00

Outline

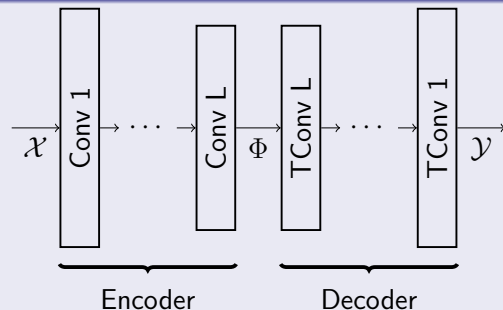
- 1 Review of last Lecture
- 2 Sequence Processing
- 3 Recurrent Neural Networks
- 4 Gated Recurrent Neural Networks

Review of last Lecture

Four-Layer Auto-Encoder



Convolutional Auto-Encoder

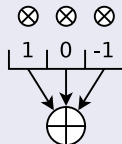


Review of last Lecture

Fractional Strided Convolution

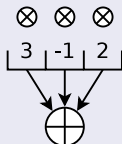
Input 0 7 3 1 6 0 5 3 0

Kernel



Output 0 -3 0 -3 0 1 0 5 0

Kernel

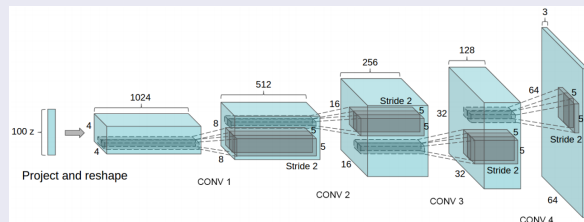


Reconstructed 3 -15 0 0 0 0 0 0 0

Generative Adversarial Network Loss

$$\mathcal{J}^{\text{GAN}} = - \sum_{n=1}^N \log \mathcal{D}(\vec{x}^{[n]}) - \sum_{n=1}^N \log (1 - \mathcal{D}(\mathcal{G}(\xi^{\vec{x}^{[n]}})))$$

Deep Convolutional Generator



Review of last Lecture

Translating Horses $\Leftrightarrow$ Zebras



zebra $\rightarrow$ horse



horse $\rightarrow$ zebra

Coloring Images

BW to Color



input



output

Motivation

What we Have so Far

- Fully-connected networks process inputs $\vec{x} \in \mathbb{R}^D$
- Convolutional networks process $\vec{x} \in \mathbb{R}^D$, $\mathbf{X} \in \mathbb{R}^{D \times E}$ or $\mathcal{X} \in \mathbb{R}^{C \times D \times E}$
- What if the input size changes?

Examples of Different Input Size

- Speech recognition
 - Different speaker speed
 - Words with different lengths
- Text processing
 - Translating sentences
 - Log file processing
- Sign language recognition
 - Videos of different lengths

Further Reading

Goodfellow Chapter 10; Skansi Chapter 7

Motivation

Example: Word Autocompletion

- Given: large corpus of text
- Learn sequence of characters
- Task: predict next character

Unsupervised Learning

- No explicit labels required
 - Next character is the target
 - Can be ambiguous:
th followed by e,a,o,i,u
- ⇒ Model sequence probabilities
- ⇒ Incorporate longer context

Freely available text corpus

Wikipedia is a free, multilingual open-collaborative online encyclopedia created and maintained by a community of volunteer contributors using a wiki-based editing system. Wikipedia is the largest general reference work on the Internet, and one of the fifteen most popular websites as ranked by Alexa; it was ranked as the thirteenth most-visited. The project carries no advertisements and is hosted by the Wikimedia Foundation, an American non-profit organization funded mainly through donations.

Modeling Text

- One-hot vector representation
 - a $\Rightarrow (1, 0, 0, \dots, 0)$
 - b $\Rightarrow (0, 1, 0, \dots, 0)$
- Used as output **and as input**

Modeling Sequences

Sequence Input Encoding

- Encoding single input as $\vec{x}$
 - For example as one-hot encoding of character
- Defining sequences of S inputs: $\vec{x}^{\{1\}}, \vec{x}^{\{2\}}, \vec{x}^{\{3\}}, \dots, \vec{x}^{\{S\}}$

How to Handle Sequences?

- Not at all: Process each symbol independently
 - Cannot model ambiguities
- Take all inputs and outputs at once
- Forward processing through sequence
 - Symbol $\vec{x}^{\{s\}}$ depends on $\vec{x}^{\{1\}}, \dots, \vec{x}^{\{s-1\}}$
 - Output $\vec{z}^{\{s\}}$ depends on $\vec{x}^{\{1\}}, \dots, \vec{x}^{\{s\}}$

Convolutional Networks

Input to Convolutional Network

Treat sequence $\{\vec{x}^{\{s\}} \mid s \in [1, S]\}$ as $\mathbf{X} \in \mathbb{R}^{S \times D}$

Fixed-Size Input

- Get largest S in your dataset
- Zero padding for shorter samples
- Use convolutional network
 - Any deep network topology

Fawaz, Forestier, Weber, Idoumghar, Muller: “Deep Learning for Time Series Classification: a Review”, DMKD 2019

Global Pooling

- Variable-length input $\mathbf{X}$
- Use convolution layers
 - Outputs $\mathcal{A} \in \mathbb{R}^{Q \times S' \times M}$
- Apply global pooling
 - Reduces output to $\underline{\mathcal{A}} \in \mathbb{R}^{Q \times 1 \times M}$
 - ⇒ Fixed-size representation
- One output $\vec{z}$ per sequence

Simple Recurrent Networks

Forward Processing Network

- Can handle one input $\vec{x}^{\{s\}}$

Feedback Loops

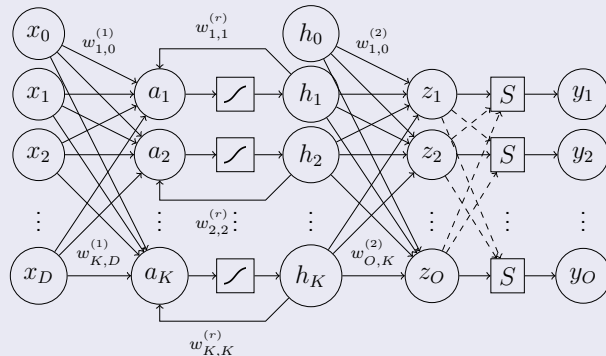
- Model dependencies

$$\vec{y}^{\{s\}} = f(\vec{x}^{\{s\}}, \vec{h}^{\{s-1\}})$$
- Output of hidden neuron = input of hidden neuron of next item in sequence:

$$\vec{a}^{\{s\}} = \mathbf{W}^{(1)} \vec{x}^{\{s\}} + \mathbf{W}^{(r)} \vec{h}^{\{s-1\}}$$

$$\vec{h}^{\{s\}} = g(\vec{a}^{\{s\}}); \quad \vec{h}^{\{0\}} = \vec{0}$$

Two-Layer Recurrent Network



Also known as Elman network

Simple Recurrent Networks

Forward Processing Network

- Can handle one input $\vec{x}^{\{s\}}$

Feedback Loops

- Model dependencies

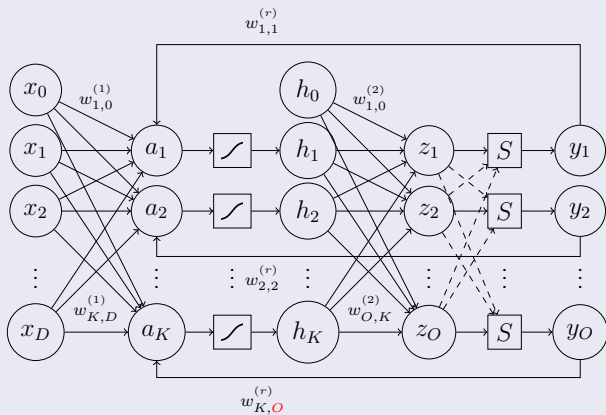
$$\vec{y}^{\{s\}} = f(\vec{x}^{\{s\}}, \vec{y}^{\{s-1\}})$$

- Output of **output** neuron = input of hidden neuron of next item in sequence:

$$\vec{a}^{\{s\}} = \mathbf{W}^{(1)} \vec{x}^{\{s\}} + \mathbf{W}^{(r)} \vec{y}^{\{s-1\}}$$

$$\vec{h}^{\{s\}} = g(\vec{a}^{\{s\}}); \quad \vec{y}^{\{0\}} = \vec{0}$$

Two-Layer Recurrent Network



Also known as **Jordan** network

Simple Recurrent Networks

Unfolding Network

- Example: input $\vec{x}^{\{1\}}, \vec{x}^{\{2\}}, \vec{x}^{\{3\}}$

Target Values

- One target per sequence:

$$\{\vec{x}^{\{1\}}, \dots, \vec{x}^{\{S\}}; \vec{t}^{\{S\}}\}$$

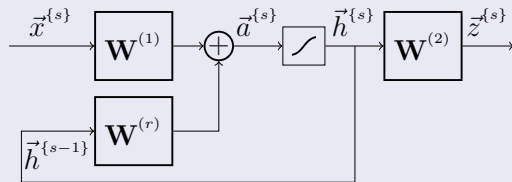
→ Example: sequence classification

- One target per sequence item:

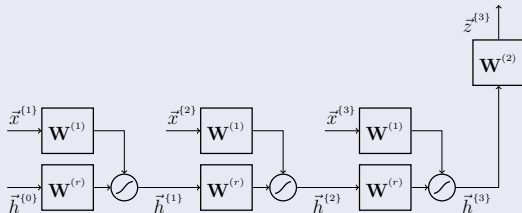
$$\{\vec{x}^{\{1\}}, \dots, \vec{x}^{\{S\}}; \vec{t}^{\{1\}}, \dots, \vec{t}^{\{S\}}\}$$

→ Example: word completion

Simplified Depiction of RNN



Unfolded Depiction of RNN



Simple Recurrent Networks

Unfolding Network

- Example: input $\vec{x}^{\{1\}}, \vec{x}^{\{2\}}, \vec{x}^{\{3\}}$

Target Values

- One target per sequence:

$$\{\vec{x}^{\{1\}}, \dots, \vec{x}^{\{S\}}; \vec{t}^{\{S\}}\}$$

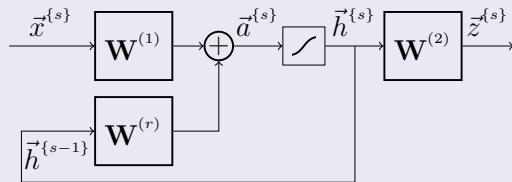
→ Example: sequence classification

- One target per sequence item:

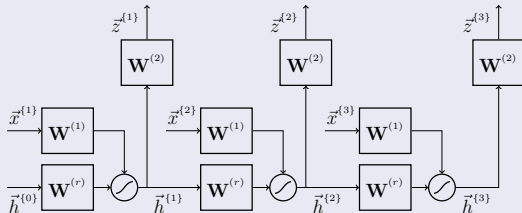
$$\{\vec{x}^{\{1\}}, \dots, \vec{x}^{\{S\}}; \vec{t}^{\{1\}}, \dots, \vec{t}^{\{S\}}\}$$

→ Example: word completion

Simplified Depiction of RNN



Unfolded Depiction of RNN



Simple Recurrent Networks

Unfolded Elman Network Output (One Target per Sequence)

$$\begin{aligned}
 \vec{y}^{\{3\}} &= g^{(2)} \left(\mathbf{W}^{(2)} \vec{h}^{\{3\}} \right) \\
 &= g^{(2)} \left(\mathbf{W}^{(2)} \left(g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{3\}} + \mathbf{W}^{(r)} \vec{h}^{\{2\}} \right) \right) \right) \\
 &= g^{(2)} \left(\mathbf{W}^{(2)} \left(g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{3\}} + \mathbf{W}^{(r)} \left(g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{2\}} + \mathbf{W}^{(r)} \vec{h}^{\{1\}} \right) \right) \right) \right) \right) \\
 &= g^{(2)} \left(\mathbf{W}^{(2)} \left(g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{3\}} + \mathbf{W}^{(r)} \left(g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{2\}} + \mathbf{W}^{(r)} g^{(1)} \left(\mathbf{W}^{(1)} \vec{x}^{\{1\}} \right) \right) \right) \right) \right) \right)
 \end{aligned}$$

General Elman Network Processing

$$\vec{y}^{\{S\}} = g^{(2)} \left(f^{(2)} \left(g^{(1)} \left(f^{(1)} \left(\vec{x}^{\{S\}}, g^{(1)} \left(f^{(1)} \left(\vec{x}^{\{S-1\}}, g^{(1)} \left(\dots \right) \right) \right) \right) \right) \right) \right)$$

Simple Recurrent Networks

Required Gradients

- Computing loss $\mathcal{J}$ between output $\vec{y}^{\{S\}}$ and target $\vec{t}^{\{S\}}$
- Simple: gradient for second layer weights

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{J}}{\partial \vec{y}^{\{S\}}} \frac{\partial \vec{y}^{\{S\}}}{\partial \vec{z}^{\{S\}}} \frac{\partial \vec{z}^{\{S\}}}{\partial \mathbf{W}^{(2)}}$$

- Complicated: gradients for first layer and recursive weights

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = ?? \quad \frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(r)}} = ??$$

- How to compute gradients for reoccurring weights?
→ Back Propagation Through Time (BPTT)

Backpropagation Through Time

Gradient for First Layer (for $g^{(1)} = \sigma$)

$$\begin{aligned}
 \frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} &= \frac{\partial \mathcal{J}}{\partial \vec{y}^{\{S\}}} \frac{\partial \vec{y}^{\{S\}}}{\partial \vec{z}^{\{S\}}} \frac{\partial \vec{z}^{\{S\}}}{\partial \vec{h}^{\{S\}}} \frac{\partial \vec{h}^{\{S\}}}{\partial \vec{a}^{\{S\}}} \frac{\partial \vec{a}^{\{S\}}}{\partial \mathbf{W}^{(1)}} \\
 &+ \frac{\partial \mathcal{J}}{\partial \vec{y}^{\{S\}}} \frac{\partial \vec{y}^{\{S\}}}{\partial \vec{z}^{\{S\}}} \frac{\partial \vec{z}^{\{S\}}}{\partial \vec{h}^{\{S\}}} \frac{\partial \vec{h}^{\{S\}}}{\partial \vec{a}^{\{S\}}} \frac{\partial \vec{a}^{\{S\}}}{\partial \vec{h}^{\{S-1\}}} \frac{\partial \vec{h}^{\{S-1\}}}{\partial \vec{a}^{\{S-1\}}} \frac{\partial \vec{a}^{\{S-1\}}}{\partial \mathbf{W}^{(1)}} \\
 &+ \frac{\partial \mathcal{J}}{\partial \vec{y}^{\{S\}}} \frac{\partial \vec{y}^{\{S\}}}{\partial \vec{z}^{\{S\}}} \frac{\partial \vec{z}^{\{S\}}}{\partial \vec{h}^{\{S\}}} \frac{\partial \vec{h}^{\{S\}}}{\partial \vec{a}^{\{S\}}} \frac{\partial \vec{a}^{\{S\}}}{\partial \vec{h}^{\{S-1\}}} \frac{\partial \vec{h}^{\{S-1\}}}{\partial \vec{a}^{\{S-1\}}} \frac{\partial \vec{a}^{\{S-1\}}}{\partial \vec{h}^{\{S-2\}}} \frac{\partial \vec{h}^{\{S-2\}}}{\partial \vec{a}^{\{S-2\}}} \frac{\partial \vec{a}^{\{S-2\}}}{\partial \mathbf{W}^{(1)}} \\
 &= (\vec{y}^{\{S\}} - \vec{t}^{\{S\}}) \mathbf{W}^{(2)} \vec{h}^{\{S\}} (1 - \vec{h}^{\{S\}}) \left(\vec{x}^{\{S\}} + \mathbf{W}^{(r)} \vec{h}^{\{S-1\}} (1 - \vec{h}^{\{S-1\}}) \left(\right. \right. \\
 &\quad \left. \left. \vec{x}^{\{S-1\}} + \mathbf{W}^{(r)} \vec{h}^{\{S-2\}} (1 - \vec{h}^{\{S-2\}}) (\vec{x}^{\{S-2\}} + \dots) \right) \right)
 \end{aligned}$$

Backpropagation Through Time

Gradient for First Layer (for $g^{(1)} = \sigma$)

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = (\vec{y}^{\{S\}} - \vec{t}^{\{S\}}) \mathbf{W}^{(2)} \sum_{s=1}^S \left(\mathbf{W}^{(r)} \right)^{S-s} \vec{x}^{\{s\}} \sum_{s'=s}^S \vec{h}^{\{s'\}} (1 - \vec{h}^{\{s'\}})$$

Observations

- $\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}}$ includes $\mathbf{W}^{(r)}$ several times
→ $\mathbf{W}^{(r)}$ is multiplied by itself
⇒ Up to $S - 1$ times: $\left(\mathbf{W}^{(r)} \right)^{S-1}$
- Derivative of $\sigma(\vec{a})$ up to S times
⇒ Vanishing gradients
- Learns only little from $\vec{x}^{\{1\}}$

Possible Improvements

- Use tanh or ReLU activation

Further Observations

- Situation better for case of one target per sequence item
- $\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(r)}}$ similar to $\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}}$
→ $\mathbf{W}^{(1)}$ is multiplied by itself

Example: Gender Classification from Name

Dataset from UCI

- Input: first names
- Target: female or male
- Many languages, contradictory
→ **Andrea** male and female
- Total: 146'752 names
→ 61 % female

Input Encoding

- Lower-case letters only
- One-hot for 26 letters and ' _ '

Network Topology

- First layer $\mathbf{W}^{(1)} \in \mathbb{R}^{1000 \times 27}$
- Recurrent $\mathbf{W}^{(r)} \in \mathbb{R}^{1000 \times 1000}$
- Second layer $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times 1000}$
- Different choices of $g^{(1)}$

Speed-Up by Parallelization

- Fixed input size $S = 25$
- Shorter names
→ prepend with $\vec{0}$

<http://archive.ics.uci.edu/ml/datasets/Gender+by+Name>

Example: Gender Classification from Name

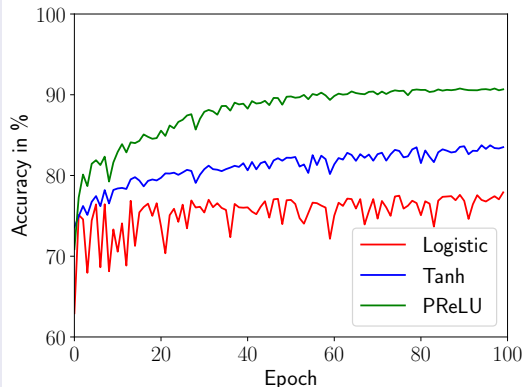
Training Setup

- Stochastic gradient descent
 - Learning rate $\eta = 0.01$
 - Momentum $\mu = 0.9$
 - Batch size $B = 128$
- Training 100 epochs
- Evaluation: training set accuracy

Observations

- Logistic activation lowest
- Tanh only little better
- PReLU continues learning

Training Set Accuracy



Long Short-Term Memory

Long-Term Dependencies

- $\vec{z}^{\{s\}}$ depends on $\vec{x}^{\{s'\}}$ with $s' \ll s$
- Example: language translation
→ Present perfect tense constructed differently in English and German

Translating English to German

- English: $\vec{x}$ = "Alice has visited Bob in his apartment."
- German: $\vec{z}$ = "Alice hat Bob in seiner Wohnung besucht."

Problem

- Difficult to learn dependency:

$$\vec{z}^{\{s\}} \sim \left(\mathbf{W}^{(r)} \right)^{s-s'} \vec{x}^{\{s'\}}$$

- $\mathbf{W}^{(r)}$ needs some path to forward $\vec{x}^{\{s'\}}$ many times

Solution

- Model dependencies explicitly
→ Memory cells in LSTM

Further Reading

Goodfellow Sections 10.7, 10.10

Long Short-Term Memory

Elements of LSTM Block

- LSTM block replaces simple recurrent matrix $\mathbf{W}^{(r)}$
- Additional memory cell $\vec{c} \in \mathbb{R}^K$ stores “latent information”
- Inputs to LSTM: $\vec{x}^{\{s\}}$, hidden node $\vec{h}^{\{s-1\}}$, memory $\vec{c}^{\{s-1\}}$
- Outputs of LSTM: updated memory $\vec{c}^{\{s\}}$, hidden node $\vec{h}^{\{s\}}$
 - Model changes of outputs through input, forget, and output gate
- Use gating mechanisms learned from input data:
 - **What** to add: $\tanh\left(\mathbf{W}^{(\cdot x)} \vec{x}^{\{s\}} + \mathbf{W}^{(\cdot h)} \vec{h}^{\{s-1\}}\right)$
 - **How much** to add: $\sigma\left(\mathbf{W}^{(\cdot x)} \vec{x}^{\{s\}} + \mathbf{W}^{(\cdot h)} \vec{h}^{\{s-1\}}\right)$
 - Weight matrices $\mathbf{W}^{(\cdot x)} \in \mathbb{R}^{K \times D}$ for input $\vec{x}^{\{s\}}$, $\mathbf{W}^{(\cdot h)} \in \mathbb{R}^{K \times K}$ for $\vec{h}^{\{s-1\}}$

Long Short-Term Memory

LSTM Inputs

- Memory cell $\vec{c}^{\{s-1\}} \in \mathbb{R}^K$
- Hidden node $\vec{h}^{\{s-1\}} \in \mathbb{R}^K$
- Input $\vec{x}^{\{s\}} \in \mathbb{R}^D$

LSTM Outputs

- Memory cell $\vec{c}^{\{s\}} \in \mathbb{R}^K$
- Hidden node $\vec{h}^{\{s\}} \in \mathbb{R}^K$

Output Gate

Influence of memory $\vec{c}^{\{s\}}$ to the output $\vec{h}^{\{s\}}$:

$$\vec{h}^{\{s\}} = \tanh(\vec{c}^{\{s\}}) \odot \sigma(\mathbf{W}^{(ox)} \vec{x}^{\{s\}} + \mathbf{W}^{(oh)} \vec{h}^{\{s-1\}})$$

Input Gate

Influence of input $\vec{x}^{\{s\}}$ and $\vec{h}^{\{s-1\}}$ to memory $\vec{c}^{\{s\}}$:

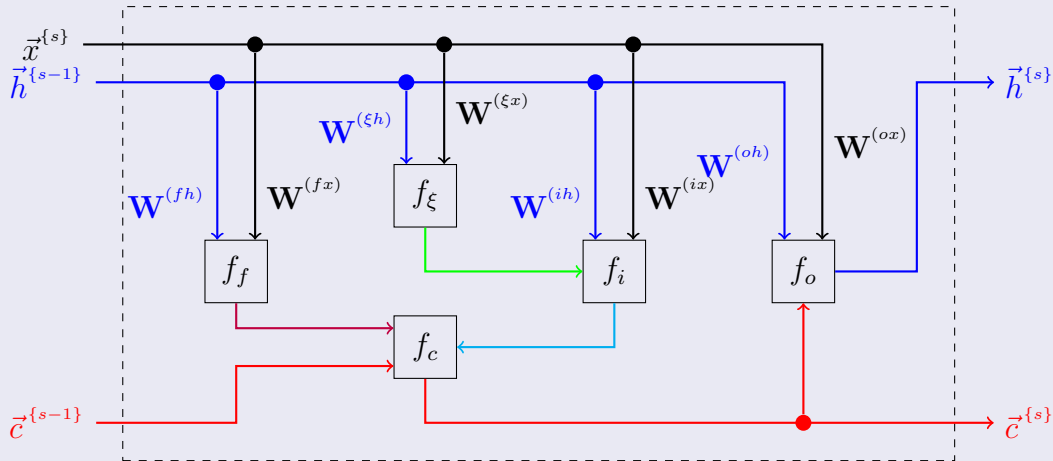
$$\begin{aligned} \xi^{\{s\}} &= \tanh(\mathbf{W}^{(ix)} \vec{x}^{\{s\}} + \mathbf{W}^{(ih)} \vec{h}^{\{s-1\}}) \\ &\odot \sigma(\mathbf{W}^{(\xi x)} \vec{x}^{\{s\}} + \mathbf{W}^{(\xi h)} \vec{h}^{\{s-1\}}) \end{aligned}$$

Forget Gate

$$\text{Change of memory } \vec{c}^{\{s\}}: \vec{c}^{\{s\}} = \xi^{\{s\}} + \vec{c}^{\{s-1\}} \odot \sigma(\mathbf{W}^{(fx)} \vec{x}^{\{s\}} + \mathbf{W}^{(fh)} \vec{h}^{\{s-1\}})$$

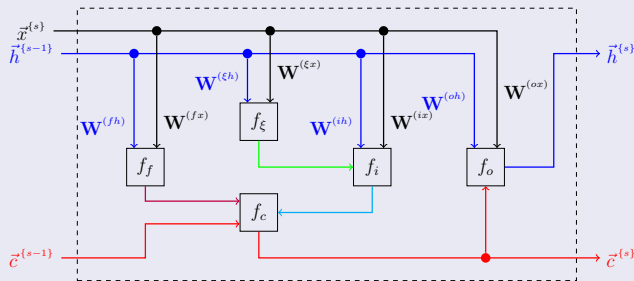
Long Short-Term Memory

Long Short-Term Memory Block – Unfolded Depiction



Long Short-Term Memory

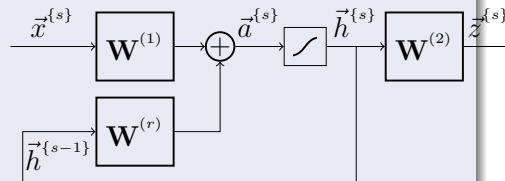
LSTM Block



Properties

- May store $c_k^{\{s'\}}$ for long time
- Learns dependencies of inputs

Simplified Depiction of RNN



Usage of LSTM Block

- Replaces $W^{(r)}$ in RNN
→ 8 matrices $W^{(\cdot x)}$, $W^{(\cdot h)}$
- Initialize $\vec{h}^{\{0\}} = \vec{c}^{\{0\}} = \vec{0}$
- Train via BPTT

Example: Gender Classification from Name

RNN Topology

- First layer $\mathbf{W}^{(1)} \in \mathbb{R}^{1000 \times 27}$
- Recurrent $\mathbf{W}^{(r)} \in \mathbb{R}^{1000 \times 1000}$
- Second layer $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times 1000}$

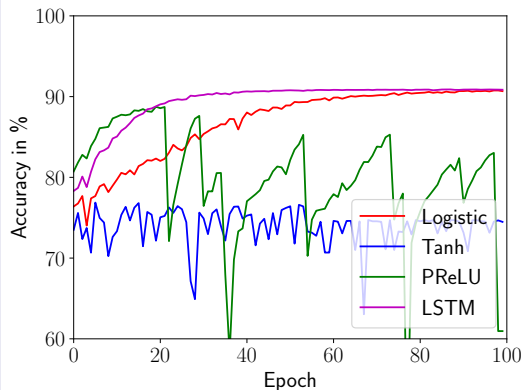
LSTM Topology

- LSTM cell with $K = 1000$
→ Last sequence element $\vec{h}^{\{S\}}$
- Second layer $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times 1000}$

Observation

- LSTM learns rather slow
→ Adam better, but unstable

Training Set Accuracy (Adam)



Variations of RNNs

Convolutional LSTM

- Applicable to image sequence processing
 - Video sequences
 - Images over time, e.g., satellite imagery
- Replaces fully-connected layers with convolutions:

$$\mathbf{W}^{(ix)} \vec{x}^{\{s\}} + \mathbf{W}^{(ih)} \vec{h}^{\{s-1\}} \Rightarrow \mathcal{X}^{\{s\}} * \mathcal{W}^{(ix)} + \mathcal{H}^{\{s-1\}} * \mathcal{W}^{(ih)}$$

Shi, Chen, Wang, Yeung, Wong, Woo: "Convolutional LSTM network: A machine learning approach for precipitation nowcasting", NeurIPS 2015

Variations of RNNs

Review of LSTM Block

- Learns 8 different matrices
- Requires 3 gates
→ Output, input, forget
- Stores data in memory cell

Gated Recurrent Units (GRU)

- Removes memory cell $\vec{c}^{\{s\}}$
- Only 2 gates: reset, update

Learnable Parameters

$$\mathbf{W}^{(rx)}, \mathbf{W}^{(rh)}, \mathbf{W}^{(ux)}, \mathbf{W}^{(uh)}, \mathbf{W}^{(hx)}, \mathbf{W}^{(hh)} \text{ — } \mathbf{W}^{(\cdot x)} \in \mathbb{R}^{K \times D}, \mathbf{W}^{(\cdot h)} \in \mathbb{R}^{K \times K}$$

Reset Gate

$$\vec{r}^{\{s\}} = \sigma\left(\mathbf{W}^{(rx)} \vec{x}^{\{s\}} + \mathbf{W}^{(rh)} \vec{h}^{\{s-1\}}\right) \odot \vec{h}^{\{s-1\}}$$

Update Gate

$$\vec{u}^{\{s\}} = \sigma\left(\mathbf{W}^{(ux)} \vec{x}^{\{s\}} + \mathbf{W}^{(uh)} \vec{h}^{\{s-1\}}\right)$$

Computing the Output

$$\begin{aligned} \vec{h}^{\{s\}} &= \left(\vec{1} - \vec{u}^{\{s\}}\right) \odot \vec{h}^{\{s-1\}} \\ &+ \vec{u}^{\{s\}} \odot \tanh\left(\mathbf{W}^{(hx)} \vec{x}^{\{s\}} + \mathbf{W}^{(hh)} \vec{r}^{\{s\}}\right) \end{aligned}$$

Variations of RNNs

Sequence Dependence

- $\vec{z}^{\{s\}}$ depends on $\vec{x}^{\{s'\}}$ with $s' \leq s$
- What if the other way round:
→ $\vec{z}^{\{s\}}$ depends on $\vec{x}^{\{s'\}}$ with $s' > s$
- Example: language translation

Translating German to English

- German: $\vec{x}$ = "Alice hat Bob in seiner Wohnung besucht."
- English: $\vec{z}$ = "Alice has visited Bob in his apartment."

Bidirectional RNNs

- Combination of two RNNs
→ Going forward through $\vec{x}^{\{s\}}$:

$$\vec{h}^{\{s\}} = \tanh\left(\mathbf{W}^{(\vec{l})} \vec{x}^{\{s\}} + \mathbf{W}^{(\vec{r})} \vec{h}^{\{s-1\}}\right)$$

- Going backward through $\vec{x}^{\{s\}}$:

$$\overleftarrow{h}^{\{s\}} = \tanh\left(\mathbf{W}^{(\overleftarrow{l})} \vec{x}^{\{s\}} + \mathbf{W}^{(\overleftarrow{r})} \overleftarrow{h}^{\{s+1\}}\right)$$

- $\vec{z}^{\{s\}}$ combines $\vec{h}^{\{s\}}$ and $\overleftarrow{h}^{\{s\}}$

Variations of RNNs

Deep RNNs

- Shallow RNNs not powerful
- Deeper networks
 - Layers $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \mathbf{W}^{(3)}, \dots$
- Stacking RNN layers
 - Layers $\mathbf{W}^{(r1)}, \mathbf{W}^{(r2)}, \mathbf{W}^{(r3)}, \dots$
 - Multiple LSTM/GRU layers

Issues with Deep RNNs

- Vanishing gradients
 - Use ReLU activations
 - Introduce skip connections

Further Reading

Goodfellow Section 10.5

Variations of RNNs

Problems of RNNs

- Lengths of source and target sequences can be different:
 $\vec{x}^{\{s_x\}}; s_x \in [1, S_x] \rightarrow \vec{y}^{\{s_y\}}; s_y \in [1, S_y]$
- Examples:
 - Text translation
 - Document summarization
- Source and target might be different types:
 - Speech to text
 - Image captioning

Sequence-to-Sequence Modeling

- Encoder and decoder networks
- Encoder encodes $\vec{x}^{\{s_x\}}$ to $\vec{\varphi}$
 - Often: $\vec{\varphi} = \vec{h}^{\{s_x\}}$ or $\vec{\varphi} = \frac{1}{S_x} \sum_{s_x} \vec{h}^{\{s_x\}}$
 - Encoding also from non-sequences
- Decoder decodes $\vec{\varphi}$ to $\vec{y}^{\{s_y\}}$
 - Decoding also to non-sequences
- Trained jointly

Further Reading

Goodfellow Section 10.4

Variations of RNNs

Auto-regressive Decoding of Sequences with RNNs

- Given: deep features $\vec{\varphi}$ from encoder
- Task: decode to sequence $\{\vec{y}^{\{s_y\}} \mid s_y \in [1, S_y]\}$
- Model: RNN with hidden state $\vec{h}^{\{s_y-1\}}$ and input $\vec{x}^{\{s_y\}}$
 - Similar modeling applies to LSTM and GRU
- Initialize $\vec{h}^{\{0\}} = \vec{\varphi}$ and $\vec{x}^{\{1\}} = \vec{0}$
- Iterate for $s_y \in \{1, \dots, S_y\}$:
 - Update hidden state: $\vec{h}^{\{s_y\}} = g^{(1)}(\mathbf{W}^{(1)}\vec{x}^{\{s_y\}} + \mathbf{W}^{(r)}\vec{h}^{\{s_y-1\}})$
 - Predict next item: $\vec{y}^{\{s_y\}} = g^{(2)}(\mathbf{W}^{(2)}\vec{h}^{\{s_y\}})$
 - Use prediction as next input: $\vec{x}^{\{s_y+1\}} = \vec{y}^{\{s_y\}}$

Variations of RNNs

Attention Gates

- Importance of sequence items
- Compute attention score $a^{\{s\}}$

Variation of Bidirectional RNN

- Similarities to LSTM/GRU
- One single attention gate
 - Additional weights $\vec{w}^{(\vec{a})}$ and $\vec{w}^{(\overleftarrow{a})}$
- Trained via BPTT

Pei, Baltrušaitis, Tax, Morency: “Temporal Attention-Gated Model for Robust Sequence Classification”, CVPR 2017

Attention Gate

$$a^{\{s\}} = \sigma\left(\vec{w}^{(\vec{a})\text{T}}\vec{h}^{\{s\}} + \vec{w}^{(\overleftarrow{a})\text{T}}\overleftarrow{h}^{\{s\}}\right)$$

Computing the Output

$$\begin{aligned}\vec{h}^{\{s\}} &= (1 - a^{\{s\}})\vec{h}^{\{s-1\}} \\ &\quad + a^{\{s\}} \tanh\left(\mathbf{W}^{(1)}\vec{x}^{\{s\}} + \mathbf{W}^{(r)}\vec{h}^{\{s-1\}}\right)\end{aligned}$$

$$\begin{aligned}\overleftarrow{h}^{\{s\}} &= (1 - a^{\{s\}})\overleftarrow{h}^{\{s+1\}} \\ &\quad + a^{\{s\}} \tanh\left(\mathbf{W}^{(1)}\vec{x}^{\{s\}} + \mathbf{W}^{(r)}\overleftarrow{h}^{\{s+1\}}\right)\end{aligned}$$